



UE23CS352A: Machine Learning Lab

Week 12: Naive Bayes Classifier

NAME: NIKITA KOLATHAYA

SRN: PES2UG23CS387

SECTION: F

DATE: 30/10/2025

INTRODUCTION:

This lab focuses on implementing and evaluating probabilistic classification techniques using the Naive Bayes algorithm.

Part A - Custom Implementation: Implemented a Multinomial Naive Bayes classifier from scratch.

Part B - Hyperparameter Optimization: Utilized scikit-learn's MultinomialNB with TF-IDF vectorization and performed systematic hyperparameter tuning using GridSearchCV to identify optimal model configurations for text classification.

Part C - Bayes Optimal Classifier: Approximated the theoretical Bayes Optimal Classifier using an ensemble of five diverse machine learning models (Naive Bayes, Logistic Regression, Random Forest, Decision Tree, and K-Nearest Neighbors) weighted by calculated posterior probabilities.

METHODOLOGY:

The Multinomial Naive Bayes approach calculates class probabilities by applying Bayes' theorem: $P(C|X) \propto P(C) \times \prod P(w_i|C)$.

To prevent numerical underflow when multiplying small probabilities, logarithmic transformations were applied throughout the calculation.

Part A: Custom Implementation

- A Multinomial Naive Bayes classifier was developed from scratch utilizing count-based feature representation.
- The implementation incorporated Laplace smoothing with $\alpha=1.0$ to handle zero-frequency problems when words not seen during training appear in test data.
- Text preprocessing included converting to lowercase and removing stop words before vectorization.

Part B: Hyperparameter Optimization

- This section employed TF-IDF (Term Frequency-Inverse Document Frequency) vectorization for feature extraction, which weights terms based on their importance across the document.
- Sklearn's MultinomialNB classifier was integrated into a GridSearchCV pipeline that systematically tested different parameter combinations.
- The grid search evaluated hyperparameters including n-gram ranges and alpha smoothing values to identify the optimal configuration using 3-fold cross-validation

Part C: Bayes Optimal Classifier Construction

- An ensemble-based Bayes Optimal Classifier was constructed using five diverse base algorithms: Naive Bayes, Logistic Regression, Random Forest, Decision Tree, and K-Nearest Neighbors.
- The posterior weights for each model were computed by splitting the training data into sub-training and validation sets, training each model on the sub-training portion, then calculating validation log-likelihoods based on predicted probabilities.
- These log-likelihoods were normalized using softmax to obtain final posterior weights. A soft voting mechanism aggregated probability predictions from all models, weighted by their respective posterior probabilities, to produce the final classification decision.

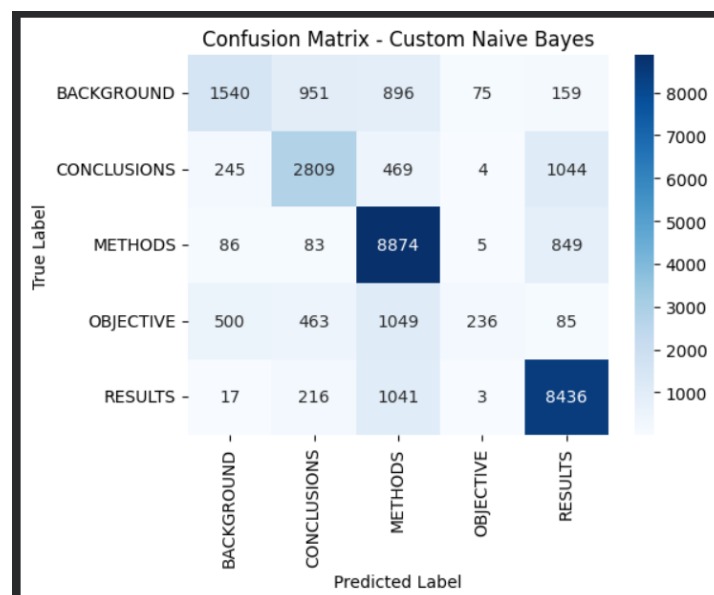
Results and Analysis (Screenshots of plots and metrics):

Part A: Screenshot of final test Accuracy, F1 Score and Confusion Matrix.

```
=== Test Set Evaluation (Custom Count-Based Naive Bayes) ===
Accuracy: 0.7483
```

	precision	recall	f1-score	support
BACKGROUND	0.54	0.57	0.55	3621
CONCLUSIONS	0.61	0.70	0.66	4571
METHODS	0.83	0.85	0.84	9897
OBJECTIVE	0.53	0.51	0.52	2333
RESULTS	0.88	0.78	0.83	9713
accuracy			0.75	30135
macro avg	0.68	0.69	0.68	30135
weighted avg	0.76	0.75	0.75	30135

Macro-averaged F1 score: 0.6809



Part B: Screenshot of best hyperparameters found and their resulting F1 score.

```
Training initial Naive Bayes pipeline...
Training complete.

=== Test Set Evaluation (Initial Sklearn Model) ===
Accuracy: 0.7266
      precision    recall  f1-score   support

 BACKGROUND      0.64      0.43      0.51      3621
 CONCLUSIONS  0.62      0.61      0.62      4571
  METHODS        0.72      0.90      0.80      9897
 OBJECTIVE       0.73      0.10      0.18      2333
  RESULTS        0.80      0.87      0.83      9713

 accuracy
macro avg      0.70      0.58      0.73      30135
weighted avg   0.72      0.73      0.70      30135

Macro-averaged F1 score: 0.5877

Starting Hyperparameter Tuning on Development Set...
Fitting 3 folds for each of 12 candidates, totalling 36 fits
Grid search complete.

Best Parameters: {'nb__alpha': 0.1, 'tfidf__ngram_range': (2, 2)}
Best Cross-Validation F1 Score: 0.6581
```

Part C:

1. Screenshot of SRN and sample size.
2. Screenshot of BOC final Accuracy, F1 Score and Confusion Matrix.

```
Please enter your full SRN (e.g., PES1UG22CS345): PES2UG23CS387
Using dynamic sample size: 10387
Actual sampled training set size used: 10387

Training all base models on sub-training set for posterior weight calculation...
Training NaiveBayes on sub-training set...
NaiveBayes validation log-likelihood: -1984.8825
Training LogisticRegression on sub-training set...
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From
warnings.warn(
LogisticRegression validation log-likelihood: -1872.5938
Training RandomForest on sub-training set...
RandomForest validation log-likelihood: -2095.5094
Training DecisionTree on sub-training set...
DecisionTree validation log-likelihood: -2607.2480
Training KNN on sub-training set...
KNN validation log-likelihood: -3012.1393

Posterior Weights: [1.71261304e-049 1.00000000e+000 1.54516809e-097 8.78448718e-320
0.00000000e+000]

Retraining all base models on full sampled training set...
Training NaiveBayes on full sampled training set...
Training LogisticRegression on full sampled training set...
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From
warnings.warn(
Training RandomForest on full sampled training set...
Training DecisionTree on full sampled training set...
Training KNN on full sampled training set...
```

```

All base models trained.

Fitting the VotingClassifier (BOC approximation)...
Fitting complete.

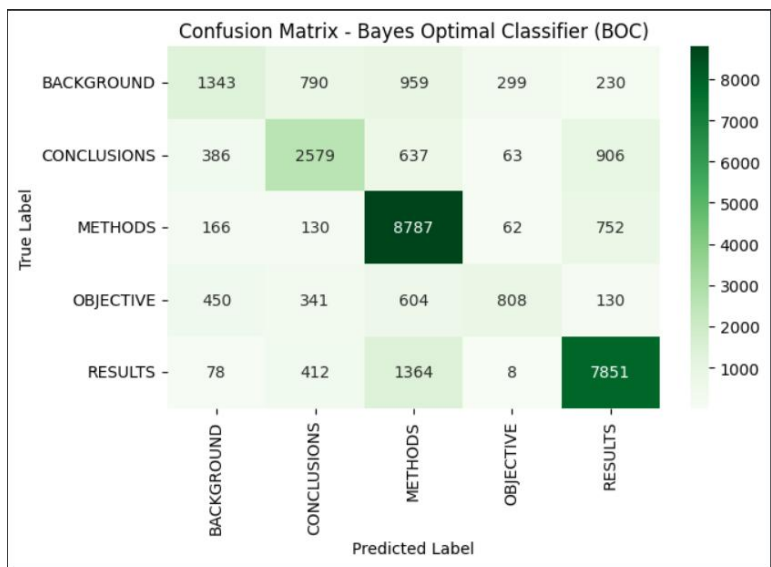
Predicting on test set...

=== Final Evaluation: Bayes Optimal Classifier (Soft Voting) ===
Accuracy: 0.7091
Macro F1 Score: 0.6146

Classification Report:

```

	precision	recall	f1-score	support
BACKGROUND	0.55	0.37	0.44	3621
CONCLUSIONS	0.61	0.56	0.58	4571
METHODS	0.71	0.89	0.79	9897
OBJECTIVE	0.65	0.35	0.45	2333
RESULTS	0.80	0.81	0.80	9713
accuracy			0.71	30135
macro avg	0.66	0.60	0.61	30135
weighted avg	0.70	0.71	0.69	30135



Discussion: Compare the performance of your scratch model (Part A) vs. the tuned Sklearn model (Part B) vs. the BOC approximation (Part C).

Part A achieved the highest scores across both metrics.

The custom model used Count-based features with bigrams (1,2), which captured important word patterns effectively.

Tuned Sklearn Model showed the weakest performance despite hyperparameter tuning:

Best parameters found: alpha=0.1, ngram_range=(2,2) (bigrams only)

The tuning process selected bigrams-only, which may have discarded useful single-word information. The initial model before tuning actually had better accuracy (72.66%) than what we might expect from bigrams alone.

BOC in the middle:

The ensemble approach (Part C) performed moderately: While it didn't achieve the best overall performance, it showed more balanced predictions across classes.

Conclusion

The lab demonstrated that model complexity doesn't always guarantee better performance. The custom-built Naive Bayes classifier with feature engineering outperformed both the hyperparameter-tuned sklearn model and the sophisticated ensemble approach. This highlights the importance of understanding your data and choosing appropriate features over blindly applying complex algorithms. **However, the BOC's more balanced class-wise performance shows the value of ensemble methods for handling difficult minority classes.**